

|                                                                                                             |                                                                                                                            |                                   |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
|  <b>Marwadi University</b> | <b>Marwadi University</b><br><b>Faculty of Technology</b><br><b>Department of Information and Communication Technology</b> |                                   |
| <b>Subject: Artificial Intelligence (01CT0616)</b>                                                          | <b>Aim: To obtain the classification of various classes using k-Nearest Neighbor approach</b>                              |                                   |
| <b>Experiment No: 03</b>                                                                                    | <b>Date: 10/3/2026</b>                                                                                                     | <b>Enrollment No: 92301733059</b> |

**Aim:** To obtain the classification of various classes using k-Nearest Neighbor approach

**IDE:** Google Colab

### **Theory:**

This algorithm is used to solve the classification model problems. K-nearest neighbor or K-NN algorithm basically creates an imaginary boundary to classify the data. When new data points come in, the algorithm will try to predict that to the nearest of the boundary line. Therefore, larger k value means smother curves of separation resulting in less complex models. Whereas, smaller k value tends to over fit the data and resulting in complex models. It's very important to have the right k-value when analyzing the dataset to avoid over fitting and under fitting of the dataset.

The model representation for KNN is the entire training dataset. It is as simple as that. KNN has no model other than storing the entire dataset, so there is no learning required. Efficient implementations can store the data using complex data structures like k-d trees to make look-up and matching of new patterns during prediction efficient. Because the entire training dataset is stored, you may want to think carefully about the consistency of your training data. It might be a good idea to curate it, update it often as new data becomes available and remove erroneous and outlier data.

K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories. K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm. K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.

K-NN is a non-parametric algorithm, which means it does not make any assumption on underlying data. It is also called a lazy learner algorithm because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset. KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

**Example:** Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.

### **Methodology:**

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Pre-process the data

|                                                                                                             |                                                                                                                            |                                   |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
|  <b>Marwadi University</b> | <b>Marwadi University</b><br><b>Faculty of Technology</b><br><b>Department of Information and Communication Technology</b> |                                   |
| <b>Subject: Artificial Intelligence (01CT0616)</b>                                                          | <b>Aim: To obtain the classification of various classes using k-Nearest Neighbor approach</b>                              |                                   |
| <b>Experiment No: 03</b>                                                                                    | <b>Date: 10/3/2026</b>                                                                                                     | <b>Enrollment No: 92301733059</b> |

5. Visualize the Data
6. Separate the feature and prediction value columns
7. Select the number K of the neighbors
8. Calculate the Euclidean distance of K number of neighbors
9. Take the K nearest neighbors as per the calculated Euclidean distance.
10. Among these k neighbors, count the number of the data points in each category.
11. Assign the new data points to that category for which the number of the neighbor is maximum.
12. Our model is ready.

### **Pre Lab Exercise:**

#### **a. What is K in KNN-approach?**

- **Hyperparameter:** K represents the number of nearest neighbors the algorithm considers to make a prediction for a new data point.
- **Decision Logic:** In classification, the output is determined by a majority vote of the K neighbors, while in regression, it is the average of their values.
- **Balance:** A small K makes the model sensitive to noise (overfitting), while a large K makes it more stable but potentially less accurate by blurring local patterns (underfitting).

#### **b. Which are the types of distance metrics?**

- **Euclidean & Manhattan Distance:** These are geometric metrics used for continuous data, measuring the straight-line "as-the-crow-flies" path or the grid-based "city block" path between points.
- **Cosine Similarity & Distance:** This measures the angle between two vectors rather than their magnitude, making it ideal for high-dimensional text data and recommendation systems.
- **Minkowski & Hamming Distance:** Minkowski acts as a generalized formula for other metrics, while Hamming is specifically used for categorical data to count differences between strings.

#### **c. What can be the criteria of selection of the value of K?**

- **Square Root Rule:** A common heuristic is to set  $K = \sqrt{n}$ , where n is the total number of samples in your training dataset, providing a quick baseline for selection.
- **Elbow Method (Error Rate):** Plot the error rate (or accuracy) against various values of K and select the "elbow" point where the error stabilizes or begins to increase.
- **Domain & Data Nature:** Use an odd value for K in binary classification to prevent ties, and choose a smaller K for low-noise data to capture local patterns or a larger K to smooth out noise.

|                                                                                                             |                                                                                                                            |                                   |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
|  <b>Marwadi University</b> | <b>Marwadi University</b><br><b>Faculty of Technology</b><br><b>Department of Information and Communication Technology</b> |                                   |
| <b>Subject: Artificial Intelligence (01CT0616)</b>                                                          | <b>Aim: To obtain the classification of various classes using k-Nearest Neighbor approach</b>                              |                                   |
| <b>Experiment No: 03</b>                                                                                    | <b>Date: 10/3/2026</b>                                                                                                     | <b>Enrollment No: 92301733059</b> |

d. What are the advantages of KNN algorithm?

- **Simplicity:** KNN is a non-parametric, lazy learning algorithm that is extremely easy to implement and understand as it requires no formal training phase.
- **Versatility:** It is naturally effective for both classification and regression tasks and can handle multi-class problems without additional complexity.
- **Adaptability:** The model constantly evolves with new data; since it stores all training instances, the decision boundary adjusts immediately as new samples are added.

### Program (Code):

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
)

digits = load_digits()
X = digits.data
y = digits.target
images = digits.images

plt.figure(figsize=(8, 4))
```

|                                                                                                             |                                                                                                                            |                                   |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
|  <b>Marwadi University</b> | <b>Marwadi University</b><br><b>Faculty of Technology</b><br><b>Department of Information and Communication Technology</b> |                                   |
| <b>Subject: Artificial Intelligence (01CT0616)</b>                                                          | <b>Aim: To obtain the classification of various classes using k-Nearest Neighbor approach</b>                              |                                   |
| <b>Experiment No: 03</b>                                                                                    | <b>Date: 10/3/2026</b>                                                                                                     | <b>Enrollment No: 92301733059</b> |

```

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(f"{y[i]}")
    plt.axis("off")
plt.suptitle("Sample Digits")
plt.tight_layout()
plt.show()
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
knn_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier())
])

knn_param_grid = {
    "knn__n_neighbors": [3, 5, 7, 9, 13, 15, 20]}
rf_pipeline = Pipeline([
    ("rf", RandomForestClassifier(random_state=42))
])

rf_param_grid = {
    "rf__n_estimators": [25, 50, 100, 200, 500],
    "rf__max_depth": [None, 10, 20]}
mlp_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("mlp", MLPClassifier(
        max_iter=1000,
        random_state=42,
        early_stopping=True
    ))
])

mlp_param_grid = {
    "mlp__hidden_layer_sizes": [(64,), (128,), (64, 64)],
    "mlp__activation": ["relu", "tanh"],
    # "mlp__alpha": [0.0001, 0.001],
    "mlp__learning_rate_init": [0.001, 0.01, 1.0, 5.0, 10.0]
}

```



**Marwadi**  
University

**Marwadi University**  
**Faculty of Technology**  
**Department of Information and Communication Technology**

**Subject: Artificial  
Intelligence (01CT0616)**

**Aim: To obtain the classification of various classes using k-Nearest  
Neighbor approach**

**Experiment No: 03**

**Date: 10/3/2026**

**Enrollment No: 92301733059**

```
}  
  
#Training the model  
def train_with_timing(pipeline, param_grid):  
    start = time.time()  
    grid = GridSearchCV(  
        pipeline,  
        param_grid,  
        cv=5,  
        scoring="f1_weighted",  
        n_jobs=-1  
    )  
    grid.fit(X_train, y_train)  
    train_time = time.time() - start  
    return grid, train_time  
  
knn_grid, knn_train_time = train_with_timing(knn_pipeline, knn_param_grid)  
rf_grid, rf_train_time = train_with_timing(rf_pipeline, rf_param_grid)  
mlp_grid, mlp_train_time = train_with_timing(mlp_pipeline, mlp_param_grid)  
#Inferencing the model  
def inference_time(model):  
    start = time.time()  
    model.predict(X_test)  
    return time.time() - start  
  
knn_test_time = inference_time(knn_grid.best_estimator_)  
rf_test_time = inference_time(rf_grid.best_estimator_)  
mlp_test_time = inference_time(mlp_grid.best_estimator_)  
#Evaluation  
def evaluate_model(model):  
    y_pred = model.predict(X_test)  
    return {  
        "Accuracy": accuracy_score(y_test, y_pred),  
        "Precision": precision_score(y_test, y_pred, average="weighted"),  
        "Recall": recall_score(y_test, y_pred, average="weighted"),  
        "F1-score": f1_score(y_test, y_pred, average="weighted")  
    }  
  
knn_metrics = evaluate_model(knn_grid.best_estimator_)
```

|                                                                                                             |                                                                                                                            |                                   |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
|  <b>Marwadi University</b> | <b>Marwadi University</b><br><b>Faculty of Technology</b><br><b>Department of Information and Communication Technology</b> |                                   |
| <b>Subject: Artificial Intelligence (01CT0616)</b>                                                          | <b>Aim: To obtain the classification of various classes using k-Nearest Neighbor approach</b>                              |                                   |
| <b>Experiment No: 03</b>                                                                                    | <b>Date: 10/3/2026</b>                                                                                                     | <b>Enrollment No: 92301733059</b> |

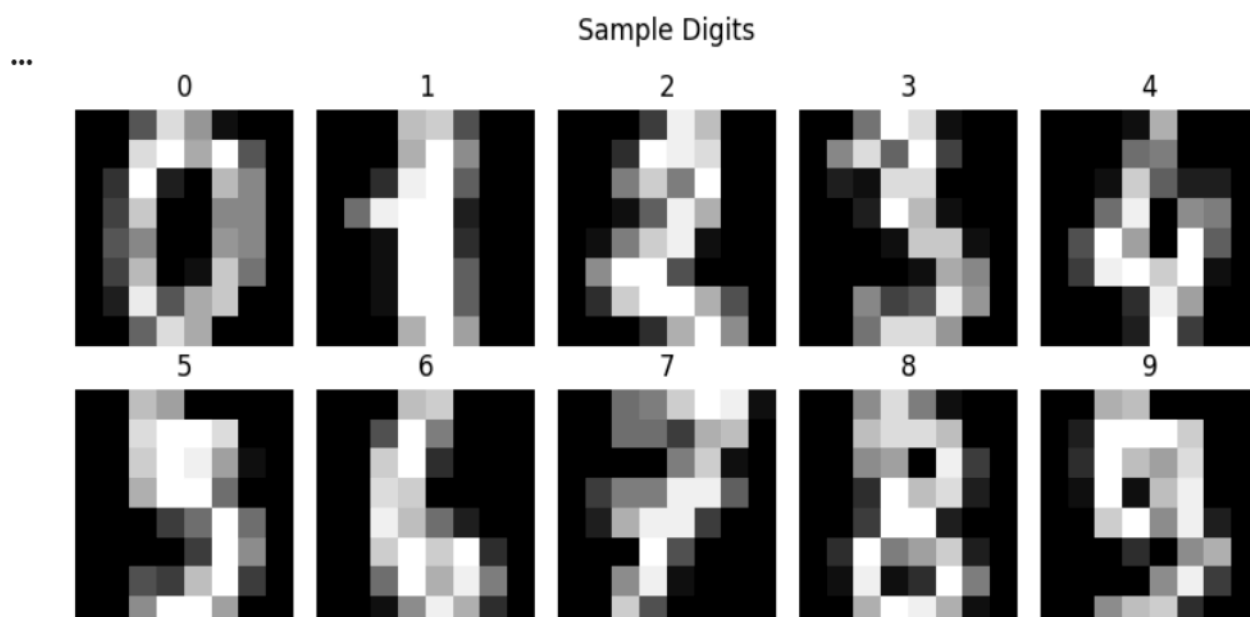
```
rf_metrics = evaluate_model(rf_grid.best_estimator_)
mlp_metrics = evaluate_model(mlp_grid.best_estimator_)
#Result with comparison
results_df = pd.DataFrame(
    [knn_metrics, rf_metrics, mlp_metrics],
    index=["KNN", "Random Forest", "MLP"]
)

results_df["Training Time (s)"] = [
    knn_train_time, rf_train_time, mlp_train_time
]

results_df["Inference Time (s)"] = [
    knn_test_time, rf_test_time, mlp_test_time
]

print(results_df)
```

## Results:

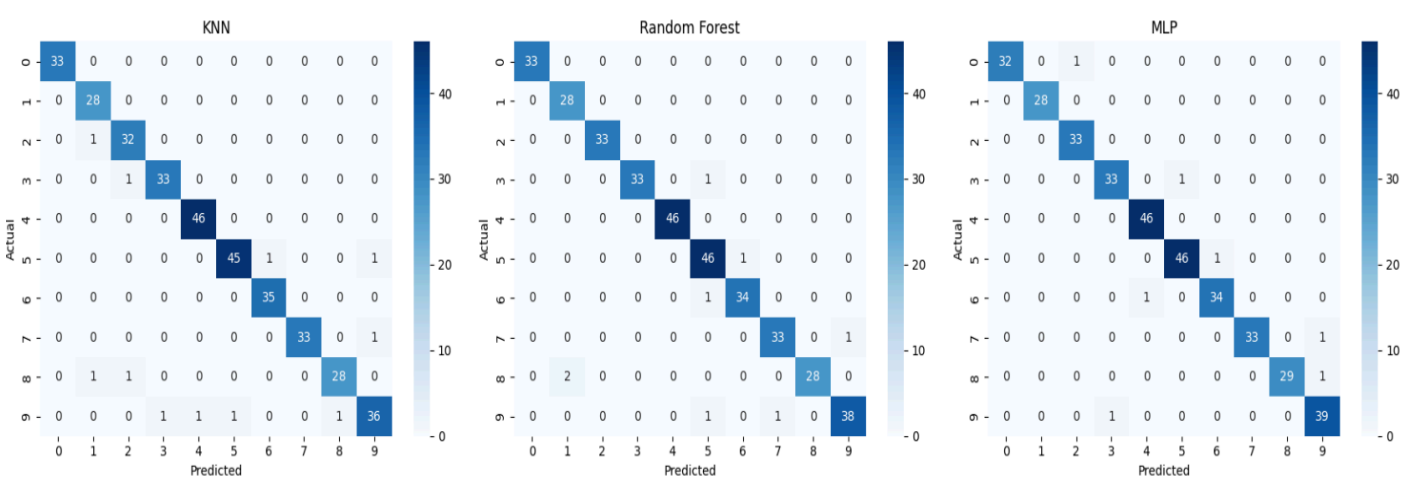
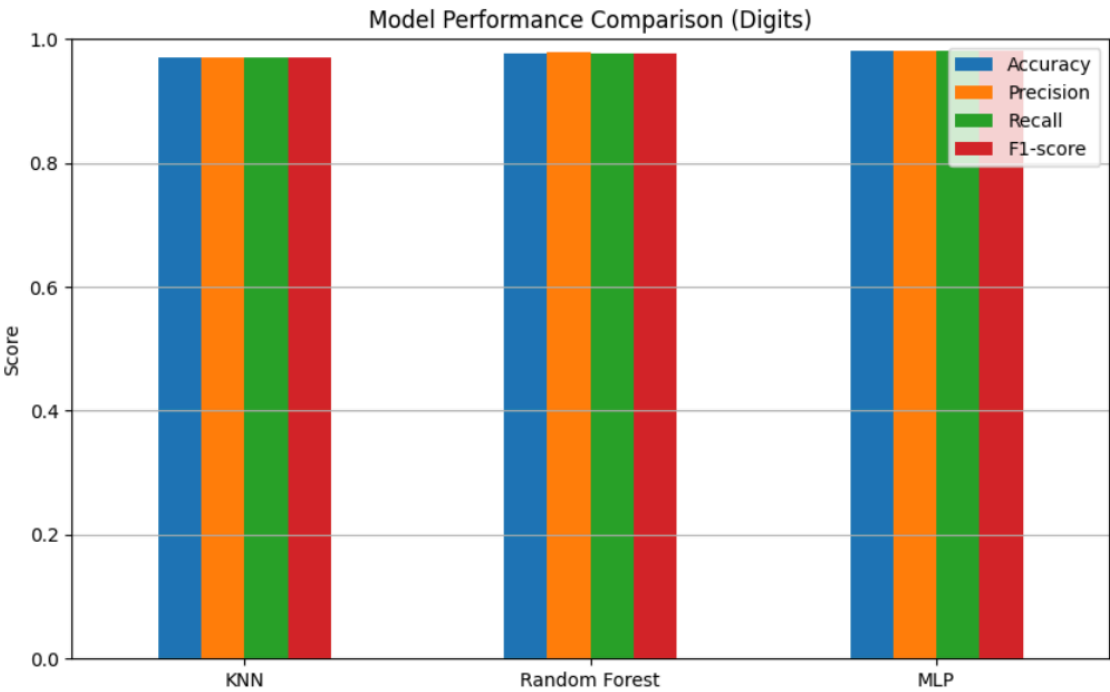


|                                                                                                                                                      |                                                                                                                            |                                                                                               |
|------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <div>  <div> <div>Marwadi</div> <div>University</div> </div> </div> | <b>Marwadi University</b><br><b>Faculty of Technology</b><br><b>Department of Information and Communication Technology</b> |                                                                                               |
|                                                                                                                                                      | <b>Subject: Artificial Intelligence (01CT0616)</b>                                                                         | <b>Aim: To obtain the classification of various classes using k-Nearest Neighbor approach</b> |
| <b>Experiment No: 03</b>                                                                                                                             | <b>Date: 10/3/2026</b>                                                                                                     | <b>Enrollment No: 92301733059</b>                                                             |

| ...           | Accuracy | Precision | Recall   | F1-score | Training Time (s) \ |
|---------------|----------|-----------|----------|----------|---------------------|
| KNN           | 0.969444 | 0.969666  | 0.969444 | 0.969287 | 3.042850            |
| Random Forest | 0.977778 | 0.978417  | 0.977778 | 0.977820 | 38.450006           |
| MLP           | 0.980556 | 0.980832  | 0.980556 | 0.980577 | 22.508983           |

|               | Inference Time (s) |
|---------------|--------------------|
| KNN           | 0.030298           |
| Random Forest | 0.048769           |
| MLP           | 0.001353           |



|                                                                                                             |                                                                                                                            |                                   |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
|  <b>Marwadi University</b> | <b>Marwadi University</b><br><b>Faculty of Technology</b><br><b>Department of Information and Communication Technology</b> |                                   |
| <b>Subject: Artificial Intelligence (01CT0616)</b>                                                          | <b>Aim: To obtain the classification of various classes using k-Nearest Neighbor approach</b>                              |                                   |
| <b>Experiment No: 03</b>                                                                                    | <b>Date: 10/3/2026</b>                                                                                                     | <b>Enrollment No: 92301733059</b> |

## Observation and Result Analysis:

### a. Nature of the dataset

- **Dimensionality:** The load\_digits dataset consists of  $8 \times 8$  pixel images (64 features). Since KNN relies on distance metrics, it performs exceptionally well here because the feature space is relatively small and the classes (0-9) are well-separated in the vector space.
- **Scaling:** As observed in the code, a StandardScaler was used. KNN is highly sensitive to the scale of features; without scaling, pixels with higher intensity variance would dominate the distance calculation.

### b. During Training Process

- The "Lazy" Nature: You will notice that KNN has a very short training time compared to Random Forest or MLP. This is because KNN does not "learn" a mathematical function; it simply stores the training vectors.
- Hyperparameter Tuning: The GridSearchCV helps in finding the optimal K. If K is too low (e.g.,  $K=1$ ), the model might capture noise in the pixel data.

### c. After the training Process

- **Computational Cost:** While KNN trained the fastest, its **Inference Time** is often higher than Random Forest. This is because, for every test image, KNN must calculate the distance to *all* 1,437 training samples to find the nearest neighbors.
- **Accuracy:** KNN often matches or outperforms complex neural networks (MLP) on this specific dataset because the "local similarity" of handwritten digits is a strong predictor of their class.

### d. Observation over the Learning Curve

- **Stability:** A smoother learning curve is observed as K increases, but there is a "sweet spot." If K becomes too large (e.g.,  $K=50$ ), the accuracy drops because the neighborhood includes too many points from other digit classes, leading to underfitting.
- **Error Rate:** The error rate typically follows a U-shape when plotted against K. The "Elbow" in the error plot justifies the selection of the best K found by your Grid Search (usually between 3 and 7 for this dataset).

## Post Lab Exercise:

### a. Is KNN useful in regression-based problems?

Yes, KNN is effective for regression. Instead of predicting a category, it calculates the average (or weighted average) of the  $K$  nearest neighbors' values to provide a continuous numerical output.

### b. Why is KNN a non-parametric algorithm?

|                                                                                                             |                                                                                                                            |                                   |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
|  <b>Marwadi University</b> | <b>Marwadi University</b><br><b>Faculty of Technology</b><br><b>Department of Information and Communication Technology</b> |                                   |
| <b>Subject: Artificial Intelligence (01CT0616)</b>                                                          | <b>Aim: To obtain the classification of various classes using k-Nearest Neighbor approach</b>                              |                                   |
| <b>Experiment No: 03</b>                                                                                    | <b>Date: 10/3/2026</b>                                                                                                     | <b>Enrollment No: 92301733059</b> |

KNN is considered non-parametric because it does not make any underlying assumptions about the distribution of the data (such as assuming the data follows a normal distribution or a linear trend).

- **No Fixed Parameters:** Unlike Linear Regression, which learns specific weights ( $\theta$ ) for a formula, KNN does not have a fixed number of parameters.
- **Data-Driven:** The model structure is determined entirely by the training data itself, allowing it to fit complex, functional forms that are not pre-defined.

### c. What are the assumptions of the KNN approach?

While it is non-parametric, KNN still relies on two fundamental assumptions:

- **Feature Similarity:** It assumes that "like" points are near each other. If two data points are close in the feature space, they likely share the same label or similar value.
- **Clean Data:** It assumes the data is relatively noise-free. Because KNN relies on local neighbors, outliers or incorrectly labeled data points can significantly "pollute" the neighborhood and lead to poor predictions.

### d. How can the KNN approach make the prediction of the unseen dataset?

When a new, unseen data point is introduced, the algorithm performs a live search:

- **Distance Calculation:** It calculates the distance (usually Euclidean) between the new point and every point in the training set.
- **Neighbor Selection:** It identifies the K points with the smallest distance values.
- **Aggregation:**
  - \* **For Classification:** It takes a majority vote of the neighbors' classes.
  - **For Regression:** It calculates the mean value of the neighbors.

### e. Why should we not use KNN for large datasets?

KNN becomes highly inefficient as the dataset grows for several reasons:

- **Computational Complexity:** For every single prediction, the algorithm must calculate the distance to *all* N points in the dataset. This is an  $O(N)$  operation per test point.
- **Memory Usage:** Since it stores the entire dataset rather than a set of learned parameters, it can consume massive amounts of RAM.
- **The "Curse of Dimensionality":** In large, high-dimensional datasets, the "distance" between points becomes almost uniform, making it difficult for the algorithm to distinguish which neighbors are truly "near."